Guía de consultas para la presentación

Proyecto Mparfum · Diseño de Bases de Datos II · Universidad de Mendoza · 2026
Cada tema de la clase *MongoDB: Gestión de Datos* (Ing. F. Zapata) aplicado a la base `mparfum`, más índices y medición de rendimiento.

Cada tema de la clase *MongoDB: Gestión de Datos* (Ing. F. Zapata) aplicado a la base `mparfum`. Los comandos se pegan **de a uno** en mongosh.

Para empezar, abrí el CMD y escribí `mongosh`. Dentro de mongosh, seleccioná la base:

```
use mparfum
```

Todo esto también corre de una sola vez con `mongosh scripts/07_consultas_clase.js`, que deja la base como estaba.

---

1. Operaciones básicas

Comando	Qué muestra
`show dbs`	Las bases del servidor (aparece `mparfum`)
`use mparfum`	Selecciona la base activa
`db`	Confirma la base activa
`show collections`	Las 10 colecciones y las 3 vistas
`db.createCollection("prueba")`	Crea una colección vacía
`db.prueba.drop()`	La elimina

Para mostrar `dropDatabase()` **sin borrar el proyecto**, usá otra base:

```
use base_prueba
db.temporal.insertOne({ x: 1 })
db.dropDatabase()
use mparfum
```

2. Insertar, consultar, modificar y eliminar

insertOne: alta de un perfume. Si falta un campo obligatorio, lo rechaza el validador.

```
db.productos.insertOne({ nombre: "Idôle", tipo: "fragancia", genero: "mujer", marca: { _id:
"lancome", nombre: "Lancôme" }, categoria: { _id: "fragancias-mujer", ruta: ["fragancias",
"fragancias-mujer"] }, etiquetas: ["floral"], variantes: [{ sku: "FR-9001", presentacion: "Eau de
Parfum 50 ml", precio: 189000, stock: [{ deposito: "DEP-MZA", cantidad: NumberInt(10) }] }],
activo: true })
```

Para mostrar el validador en acción, intentá insertar un producto mal cargado:

```
db.productos.insertOne({ nombre: "Sin datos", tipo: "perfume" })
```

→ `Document failed validation`

find / findOne

```
db.productos.find()
db.productos.findOne()
db.productos.find({ tipo: "fragancia", genero: "hombre" }, { nombre: 1, "marca.nombre": 1, _id: 0
})
db.clientes.findOne({ email: "camila.rodriguez49@yahoo.com.ar" })
```

update con `$set`, `$inc` y `$unset`

```
db.productos.updateOne({ nombre: "Idôle" }, { $set: { descripcion: "Floral chipre, lanzamiento
2026" } })
db.productos.updateOne({ nombre: "Idôle" }, { $inc: { "variantes.0.precio": 5000 } })
db.productos.updateOne({ nombre: "Idôle" }, { $unset: { descripcion: "" } })
```

Eliminar (`remove` de la clase = `deleteOne` / `deleteMany` en mongosh nuevo)

```
db.productos.deleteOne({ nombre: "Idôle" })
```

3. Gestión de arrays

Se trabaja sobre las `etiquetas` de un producto. Primero se crea uno de prueba:

```
db.productos.insertOne({ nombre: "Prueba arrays", tipo: "fragancia", marca: { _id: "dior", nombre:
"Dior" }, categoria: { _id: "fragancias-mujer", ruta: ["fragancias"] }, etiquetas: ["floral"],
variantes: [{ sku: "FR-9002", presentacion: "50 ml", precio: 1000, stock: [] }], activo: true })
```

Operador	Comando
`$push`	`db.productos.updateOne({ nombre: "Prueba arrays" }, { $push: { etiquetas: "día de la madre" } })`
`$push` + `$each`	`db.productos.updateOne({ nombre: "Prueba arrays" }, { $push: { etiquetas: { $each: ["noche", "elegante", "best seller"] } } })`
`$pop: 1` (último, pila)	`db.productos.updateOne({ nombre: "Prueba arrays" }, { $pop: { etiquetas: 1 } })`
`$pop: -1` (primero, cola)	`db.productos.updateOne({ nombre: "Prueba arrays" }, { $pop: { etiquetas: -1 } })`
`$pull`	`db.productos.updateOne({ nombre: "Prueba arrays" }, { $pull: { etiquetas: "noche" } })`
`$addToSet` (no duplica)	`db.productos.updateOne({ nombre: "Prueba arrays" }, { $addToSet: { etiquetas: "elegante" } })`

Para ver cómo quedó y después borrarlo:

```
db.productos.findOne({ nombre: "Prueba arrays" }, { etiquetas: 1 })
db.productos.deleteOne({ nombre: "Prueba arrays" })
```

En el proyecto real, los arrays se usan en `favoritos` de cada cliente (`$addToSet` / `$pull` desde la tienda web) y en `historialEstados` de cada pedido (`$push` cada vez que cambia de estado).

4. Carga masiva

```
db.prueba.insertMany([{ _id: 0 }, { _id: 1 }, { _id: 2 }, { _id: 3 }])
db.prueba.insertMany([{ _id: 10 }, { _id: 11 }, { _id: 11 }, { _id: 12 }])
```

→ Error E11000. Se insertan 10 y 11, y **el 12 no**: el proceso se interrumpe en el duplicado.

```
db.prueba.insertMany([{ _id: 20 }, { _id: 21 }, { _id: 21 }, { _id: 22 }], { ordered: false })
```

→ Con `ordered: false` saltea el duplicado y **sí inserta el 22**.

```
for (var i = 0; i < 1000; i++) { db.prueba.insertOne({ clave: "dato" + i, num: i, otro: i * 2 }) }
db.prueba.countDocuments()
db.prueba.drop()
```

En el proyecto, el script `01_carga_datos.js` carga los 20.000 pedidos con `insertMany` en lotes de 5.000, que es mucho más rápido que un `insertOne` por documento.

5. Operadores de consulta

Operador	Consulta en Mparfum
`$gte` / `$lte` (rango)	`db.productos.find({ "variantes.precio": { $gte: 100000, $lte: 150000 } }, { nombre: 1 })`
`$gt` / `$lt`	`db.pedidos.countDocuments({ total: { $gt: 1000000 } })`
`$in`	`db.productos.find({ "marca._id": { $in: ["chanel", "dior", "lancome"] } }, { nombre: 1 })`
`$nin` / `$ne`	`db.productos.find({ tipo: { $nin: ["fragancia", "set"] }, genero: { $ne: "unisex" } }, { nombre: 1 })`
`$or`	`db.productos.find({ $or: [{ genero: "hombre" }, { tipo: "cabello" }] }, { nombre: 1 })`
`$and` explícito	`db.pedidos.countDocuments({ $and: [{ canal: "app" }, { "pago.cuotas": 12 }] })`
`$exists`	`db.pedidos.countDocuments({ promocion: { $exists: true } })`
Expresión regular	`db.productos.find({ nombre: /^s/i }, { nombre: 1 })`
`$elemMatch`	`db.productos.find({ variantes: { $elemMatch: { precio: { $lt: 60000 }, "contenido.unidad": "ml" } } }, { nombre: 1 })`
`$all`	`db.productos.find({ "atributos.notas.fondo": { $all: ["vainilla", "pachulí"] } }, { nombre: 1 })`
`$size`	`db.productos.countDocuments({ variantes: { $size: 3 } })`
Notación de punto	`db.pedidos.countDocuments({ "envio.direccion.ciudad": "Godoy Cruz" })`
`$text` (índice de texto)	`db.productos.find({ $text: { $search: "vainilla" } }, { nombre: 1 })`

Operador	Consulta en Mparfum
`sort` / `skip` / `limit`	`db.productos.find({}, { nombre: 1, rating: 1 }).sort({ "rating.promedio": -1 }).skip(0).limit(5)`
`distinct`	`db.productos.distinct("atributos.familiaOlfativa")`

6. Tipos de datos

Tipo	Dónde está en Mparfum	Consulta
`string`	`nombre`, `email`	`db.clientes.countDocuments({ email: /gmail/ })`
`number` (int / double)	`numero` y `precio` son int (enteros); `rating.promedio` es double (decimales). En mongosh 2.x, `5` se guarda como int y `5.5` como double	`db.productos.countDocuments({ "rating.promedio": { $type: "double" } })`
`boolean`	`activo`, `compraVerificada`	`db.productos.find({ activo: false })`
`date`	`fecha` de pedidos, `fechaNacimiento`	`db.pedidos.countDocuments({ fecha: { $gte: new Date("2026-10-01") } })`
`array`	`variantes`, `etiquetas`, `favoritos`	`db.productos.find({ etiquetas: "best seller" }, { nombre: 1 })`
documento embebido	`marca`, `envio`, `pago`, `club`	`db.clientes.countDocuments({ "club.nivel": "black" })`
`ObjectId`	`_id`, `cliente._id`, `items.productoId`	`db.pedidos.findOne()._id.getTimestamp()`
`null` / ausente	`tono` sólo en maquillaje	`db.productos.find({ "variantes.tono": { $exists: true } }, { nombre: 1 })`
`regex`	búsquedas	`db.productos.countDocuments({ descripcion: /floral/i })`
GeoJSON (extra)	`depositos.ubicacion`	consulta C13 (`$geoNear`)

7. Relaciones

Cardinalidad	Ejemplo en Mparfum	Cómo se resolvió	Por qué
Uno a uno	Pedido → envío	Embebido (`pedido.envio`)	Siempre se leen juntos y el envío no existe sin el pedido
Uno a muchos (embebido)	Producto → variantes (30, 50, 100 ml)	Array embebido	Son pocas, se muestran juntas en la ficha y se actualiza el stock en una sola operación

Cardinalidad	Ejemplo en Mparfum	Cómo se resolvió	Por qué
Uno a muchos (referenciado)	Cliente → pedidos	`pedido.cliente._id`	Los pedidos crecen sin límite y no entrarían en el documento del cliente (16 MB)
Uno a muchos (referenciado)	Producto → reseñas	`reseña.productoId`	Ídem: crecen sin límite
Muchos a muchos	Clientes ↔ productos (favoritos)	Array de `ObjectId` en el cliente	Un cliente tiene varios favoritos y un producto está en los favoritos de muchos clientes
Muchos a uno	Productos → marca / categoría	Referencia extendida `{ _id, nombre }`	Se muestra el nombre sin hacer `$lookup`

Se usan `_id` de texto en marcas y categorías (`"dior"`, `"fragancias-mujer"`) porque son legibles y estables, y `ObjectId` en lo que se genera en cantidad (clientes, pedidos, productos). La clase pide "considerar el tipo de dato adecuado para el `_id`".

Uno a muchos con `$lookup` (el ejemplo de la clase con clientes y pedidos):

```
db.clientes.aggregate([ { $match: { email: "camila.rodriguez49@yahoo.com.ar" } }, { $lookup: {
from: "pedidos", localField: "_id", foreignField: "cliente._id", as: "pedidosDelCliente" } }, {
$project: { nombre: 1, apellido: 1, cantidad: { $size: "$pedidosDelCliente" } } } ])
```

Muchos a muchos con `$lookup`: los favoritos de un cliente con su nombre.

```
db.clientes.aggregate([ { $match: { "favoritos.2": { $exists: true } } }, { $limit: 1 }, {
$lookup: { from: "productos", localField: "favoritos", foreignField: "_id", as: "prod" } }, {
$project: { nombre: 1, favoritos: "$prod.nombre" } } ])
```

8. Consultas avanzadas con `aggregate()`

`$match` + `$group` + `$sum`: facturación por canal.

```
db.pedidos.aggregate([ { $match: { estado: { $ne: "cancelado" } } }, { $group: { _id: "$canal",
facturacion: { $sum: "$total" }, pedidos: { $sum: 1 } } } ])
```

`$group` con `_id: null`: un único total.

```
db.pedidos.aggregate([ { $match: { estado: "entregado" } }, { $group: { _id: null, suma: { $sum:
"$total" }, promedio: { $avg: "$total" } } } ])
```

`$project`: incluir y excluir campos. No se pueden mezclar 1 y 0, salvo `_id: 0`.

```
db.clientes.aggregate([ { $project: { _id: 0, apellido: 1, nombre: 1, email: 1 } } ])
db.clientes.aggregate([ { $project: { dni: 0, telefono: 0, direcciones: 0 } } ])
```

`$project`: renombrar y calcular con `$add` y `$subtract`.

```
db.pedidos.aggregate([ { $match: { descuento: { $gt: 0 } } }, { $limit: 5 }, { $project: { _id: 0,
nroPedido: "$numero", cliente: "$cliente.nombre", sinDescuento: { $add: ["$subtotal",
"$envio.costo"] }, pagado: { $subtract: [{ $add: ["$subtotal", "$envio.costo"] }, "$descuento"] }
} } ])
```

`$project` + **`$cond`**: gama de precio de cada producto.

```
db.productos.aggregate([ { $project: { nombre: 1, precio: { $min: "$variantes.precio" } } }, { $project: { nombre: 1, precio: 1, gama: { $cond: { if: { $gte: ["$precio", 150000] }, then: "Premium", else: "Accesible" } } } } ])
```

`$cond` con edades (como el Adulto / Menor de la clase):

```
db.clientes.aggregate([ { $project: { nombre: 1, edad: { $dateDiff: { startDate: "$fechaNacimiento", endDate: "$$NOW", unit: "year" } } } }, { $project: { nombre: 1, edad: 1, categoria: { $cond: { if: { $gte: ["$edad", 18] }, then: "Adulto", else: "Menor" } } } } ])
```

`$lookup` + **`$unwind`** + **`$group`** + **`$sort`**: las marcas que más facturan.

```
db.pedidos.aggregate([ { $unwind: "$items" }, { $lookup: { from: "productos", localField: "items.productoId", foreignField: "_id", as: "p" } }, { $group: { _id: { $first: "$p.marca.nombre" }, total: { $sum: { $multiply: ["$items.cantidad", "$items.precioUnitario"] } } } }, { $sort: { total: -1 } }, { $limit: 5 } ])
```

Más consultas avanzadas del proyecto (en `03_consultas.js`): `$facet` (C18), `$bucket` (C17), `$graphLookup` (C15), `$geoNear` (C13), `$setWindowFields` (C9), `$merge` (C27) y la transacción (C29).

9. Series temporales

La colección **`eventos_web`** (script 06) registra cada vista de producto, cada "agregar al carrito" y cada búsqueda del sitio.

```
db.createCollection("eventos_web", { timeseries: { timeField: "fecha", metaField: "meta", granularity: "minutes" }, expireAfterSeconds: 15552000 })
```

Encaja con el *¿cuándo usarlas?* de la clase:

- ✅ se insertan muchos datos ordenados por tiempo (cada clic);
- ✅ se consulta por rangos de tiempo (visitas por día, horas pico);
- ✅ no se modifican ni se borran a mano (expiran solos a los 180 días);
- ✅ ocupan menos disco: en la prueba, **47% menos** que la misma información en una colección común.

Visitas por día de la última semana:

```
db.eventos_web.aggregate([ { $match: { "meta.tipo": "vista_producto", fecha: { $gte: new Date(Date.now() - 7*86400000) } } }, { $group: { _id: { $dateTrunc: { date: "$fecha", unit: "day", timezone: "America/Argentina/Mendoza" } }, vistas: { $sum: 1 } } }, { $sort: { _id: 1 } } ])
```

Horas pico:

```
db.eventos_web.aggregate([ { $group: { _id: { $hour: { date: "$fecha", timezone: "America/Argentina/Mendoza" } }, eventos: { $sum: 1 } } }, { $sort: { eventos: -1 } }, { $limit: 5 } ])
```

10. Crear índices

Sintaxis: `db.coleccion.createIndex({ campo: 1 }, { opciones })`. El `1` es ascendente y el `-1` descendente.

Tipo	Comando	Para qué
Simple	`db.productos.createIndex({ "marca._id": 1 })`	Buscar por marca
Con nombre	`db.productos.createIndex({ "marca._id": 1 }, { name: "ix_marca" })`	Nombre propio en vez del automático
Único	`db.clientes.createIndex({ email: 1 }, { unique: true })`	No permite emails repetidos (error E11000)
Compuesto	`db.pedidos.createIndex({ "cliente._id": 1, fecha: -1 })`	"Mis pedidos": filtra por cliente y devuelve ordenado por fecha. Orden de los campos: regla **ESR** (Igualdad, Orden, Rango)
Multikey (array)	`db.pedidos.createIndex({ "items.productoId": 1 })`	Indexa cada elemento del array
Texto	`db.productos.createIndex({ nombre: "text", descripcion: "text" }, { default_language: "spanish" })`	Buscador con `$text`. Sólo uno por colección (en `productos` ya existe)
Parcial	`db.productos.createIndex({ "rating.promedio": -1 }, { partialFilterExpression: { activo: true } })`	Indexa sólo los productos activos
TTL	`db.carritos.createIndex({ actualizado: 1 }, { expireAfterSeconds: 2592000 })`	Borra los carritos sin actividad a los 30 días
Geoespacial	`db.depositos.createIndex({ ubicacion: "2dsphere" })`	Punto de retiro más cercano con `$geoNear`

Ver, probar y borrar

Acción	Comando
Ver los índices	`db.pedidos.getIndexes()`
Sólo los nombres	`db.pedidos.getIndexes().map(i => i.name)`
Cuántas veces se usó cada uno	`db.pedidos.aggregate([{ $indexStats: {} }, { $project: { _id: 0, nombre: "$name", usos: "$accesses.ops" } }])`
Tamaño de cada índice	`db.pedidos.stats().indexSizes`
Borrar uno	`db.productos.dropIndex("ix_marca")`

En Compass: colección → pestaña **Indexes** → botón **Create Index**.

Demostración en vivo sin romper nada (crear un índice, comprobar la mejora y borrarlo):

```
db.clientes.find({ "preferencias.tipoPiel": "grasa"
}).explain("executionStats").executionStats.totalDocsExamined
```

→ 2.000: sin índice recorre todos los clientes.

```
db.clientes.createIndex({ "preferencias.tipoPiel": 1 }, { name: "ix_prueba_piel" })
db.clientes.find({ "preferencias.tipoPiel": "grasa"
}).explain("executionStats").executionStats.totalDocsExamined
```

→ unos 400: sólo los clientes con piel grasa.

```
db.clientes.dropIndex("ix_prueba_piel")
```

11. Medir el rendimiento

Se mide con `explain("executionStats")`. Los cuatro datos para mirar:

Dato	Qué significa	Lo ideal
`stage`	`IXSCAN` = usó un índice. `COLLSCAN` = recorrió toda la colección	`IXSCAN`
`totalDocsExamined`	Documentos leídos	Lo más cerca posible de `nReturned`
`nReturned`	Documentos devueltos	—
`executionTimeMillis`	Tiempo en milisegundos	Lo más bajo posible

Función para ver el resumen (se pega una sola vez en mongosh):

```
const ver = (exp) => { const s = exp.executionStats; let p =
exp.queryPlanner.winningPlan.queryPlan || exp.queryPlanner.winningPlan; while (p.inputStage) p =
p.inputStage; return { plan: p.stage, indice: p.indexName || "ninguno", docsExaminados:
s.totalDocsExamined, devueltos: s.nReturned, ms: s.executionTimeMillis } }
```

Comparar sin índice y con índice ("mis pedidos" de un cliente):

```
const cli = db.pedidos.findOne().cliente._id
```

Sin índice (`hint({ $natural: 1 })` obliga a recorrer toda la colección):

```
ver(db.pedidos.find({ "cliente._id": cli }).sort({ fecha: -1 }).hint({ $natural: 1
}).explain("executionStats"))
```

→ `{ plan: 'COLLSCAN', indice: 'ninguno', docsExaminados: 20000, devueltos: 12, ms: 15 }`

Con índice:

```
ver(db.pedidos.find({ "cliente._id": cli }).sort({ fecha: -1 }).explain("executionStats"))
```

→ `{ plan: 'IXSCAN', indice: 'ix_cliente_fecha', docsExaminados: 12, devueltos: 12, ms: 0 }`

Otros ejemplos:

```
ver(db.pedidos.find({ numero: 105000 }).explain("executionStats"))
ver(db.productos.find({ $text: { $search: "vainilla" } }).explain("executionStats"))
ver(db.clientes.find({ email: "camila.rodriguez49@yahoo.com.ar" }).explain("executionStats"))
```

En Compass: colección → pestaña **Explain Plan** → filtro `{ numero: 105000 }` → **Explain**. Para comparar, en **Options** poner en **Hint** `{ "$natural": 1 }`.

Medición completa del proyecto (7 consultas con y sin índice), desde el CMD:

```
mongosh "scripts/04_desempeño_indices.js"
```

Consulta (200.000 pedidos)	Docs examinados sin → con	Tiempo sin → con
Mis pedidos de un cliente	200.000 → 10	121 ms → 0 ms
Pedido por número	200.000 → 1	113 ms → 0 ms
Cola de despacho del depósito	200.000 → 1.853	151 ms → 6 ms
Pedidos que incluyen un producto	200.000 → 5.527	178 ms → 11 ms
Login por email (consulta cubierta)	2.000 → 0	1 ms → 0 ms

Preguntas que puede hacer el profesor

¿Por qué MongoDB y no una base relacional? Un perfume y una base de maquillaje tienen atributos totalmente distintos. En SQL harían falta muchas tablas o columnas nulas. Con documentos, cada producto guarda lo suyo en `atributos`, y el validador `$jsonSchema` controla lo común.

¿Por qué los ítems del pedido están embebidos y no referenciados? Porque el pedido tiene que guardar el precio del momento de la compra. Si mañana sube el precio del producto, el pedido viejo no debe cambiar.

¿Por qué las reseñas no están dentro del producto? Porque crecen sin límite. Un documento no puede superar los 16 MB, y además el producto se volvería pesado de leer en el catálogo.

¿Qué colección es la más usada? `pedidos`: la más grande, la que más crece, la que más se consulta y la que tiene más índices. `productos` es la más leída por los clientes.

¿Cómo se mejoró el rendimiento? Con índices. Ver `docs/04_indices_y_desempeño.md`: "mis pedidos" pasa de revisar 200.000 documentos a revisar 10.

¿Cómo se protege la información? Con backup diario con `mongodump`, retención 7/4/12 y prueba de restauración semanal (`docs/03_politica_resguardo.md`), más roles y usuarios con mínimo privilegio (script 05).